

AI-NATIVE DEVELOPMENT FRAMEWORK

Vibewire

从 Vibe Coding 到 Vibe Wire
从需求到交付的 AI 工程流水线

2026 年 4 月

一个反复被验证的规律

每一次工程能力的跃迁，背后都是同一套思想：将复杂过程拆解为可观测、可反馈、可自动化的闭环。

- 1948 — 控制论
Norbert Wiener 提出反馈控制：系统通过感知偏差、自动纠正来维持目标状态
- 1950s — 系统动力学
Jay Forrester 用存量/流量建模复杂系统行为，揭示反馈环的决定性作用
- 2010s — DevOps / CI-CD
反馈环思想落地软件交付：Plan → Code → Build → Test → Deploy → Monitor → 循环
- 2025 — AI Coding
大模型让"用自然语言描述需求、AI 生成代码"成为可能——但闭环在哪里？

核心洞察

没有反馈闭环的系统，无论初始表现多好，终将偏离目标。AI 编码也不例外。

Vibe Coding：人人都能"说"出软件

2025 年初，Andrej Karpathy 提出一个新范式——用自然语言描述你想要的，让 AI 写代码。

84%

开发者在使用 AI 编码工具

Stack Overflow 2025 调查

29%

开发者信任 AI 生成的代码

同一份调查

-20%

22-25 岁开发者就业变化

Stanford 2022-2025 研究

令人兴奋的部分

- 门槛大幅降低——非专业开发者也能构建原型
- 速度惊人——简单功能分钟级交付
- 意图驱动——从"怎么写"转向"写什么"

被忽视的代价

- **Context Rot** — 长会话中 AI 丢失上下文，质量持续退化
- **理解债务** — 开发者发布了自己不理解的代码
- **规模化崩溃** — 项目超过一定规模后变得不可控

行业已在寻找答案

三套代表性体系，各自试图驯服 AI 编码的不可控性。



everything-claude-code

141K Stars

定位：性能优化"驯化"手册

- Skills + Instincts 配置框架
- 跨会话 Memory 持久化
- Research-first 工作流

最佳实践集 需自行组装



Superpowers

28K+ Stars

定位：方法论强制执行框架

- brainstorm → plan → execute 流水线
- 强制 TDD (RED-GREEN-REFACTOR)
- Subagent 并行 + 双阶段审查

多平台 流程较重



GSD (Get Shit Done)

47.9K Stars

定位：Spec-Driven 完整工作流

- discuss → plan → execute → verify → ship
- Wave 并行执行 + 文件外部化状态
- 元提示 + 上下文工程驱动

功能丰富 30+ 命令

三套工具，四个共识

虽然形态各异，但三套工具已经在关键维度上形成了行业共识，并各自给出了有效解法。

规划纪律

强制"先想清楚再动手"——不再让 AI 直接写代码，而是先输出计划或规格文档。

everything-claude-code · Research-first

Superpowers · brainstorm → plan → execute

GSD · discuss → plan → execute

上下文持久化

把状态从对话内存中搬出来——写入文件系统，让每个新 Agent 都能读到完整上下文。

everything-claude-code · Memory 持久化

Superpowers · 文件外部化状态

GSD · Wave 状态文件

质量门控

不再信任 AI 的第一次输出——通过测试先行、自动审查、人工审批等多重检查保障质量。

everything-claude-code · 安全 Hooks

Superpowers · 强制 TDD + 双阶段审查

GSD · 验收测试

并行执行

多个 Agent 同时工作——不再串行等待，用并行换取速度。

Superpowers · Subagent 并行

GSD · Wave 并行执行

共识已经形成——规划、持久化、质量、并行是 AI 编码工程化的四大支柱。三套工具的差异在于实现方式，而非该不该做。

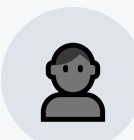
零部件齐了，机器还没转起来

三个工具各有所长，但拼不出一条从需求到交付的完整流水线。



碎片化拼装

每个工具擅长一个环节，但规划、执行、质量保障之间需要用户手动串联——没有一条自动流转的完整流水线。



人是操作者，非决策者

用户仍需触发每一步、检查状态、处理异常——减少的是工作量，不是参与度。理想状态是人只做关键决策。



质量是附加，非内建

TDD 和代码审查作为后置环节存在，而非编织在执行流程中的自动循环——审查发现问题后缺乏自动回流修复的闭环。

大项目撑不住

即使拼齐了所有环节，现有工具在面对大型项目时仍然力不从心：单会话上下文终会耗尽，跨里程碑的全局规划依赖人工维护，任务间依赖关系无法自动追溯。

根本缺失

缺少一套从需求到交付的
端到端自动化闭环

计划赶不上变化

Spec 阶段不可能预料所有情况——执行时需要根据真实情况修正，但修正无反馈到 Spec，导致文档与代码彻底脱节。



Most spec-driven tools produce static documents that drift from implementation within hours.

— Augmentcode, 2026

强行要求"文档与代码实时完美同步"会**严重拖慢开发节奏**。

— 阿里云开发者社区, 2026

根本原因：Spec 是一次性的快照

它在某个时刻对需求的理解是准确的，但执行中的发现（API 限制、性能瓶颈、框架约束）无法自动回流。所有复杂项目中，Spec 都必然存在未预见的实现细节——这不是执行偏差，而是**计划的天然局限**。

Vibe + Wire

保留 Vibe 的自然表达力，加上 Wire 的工程纪律。

Vibe

用户用自然语言描述意图，不需要关心实现细节。就像 Vibe Coding 一样简单——说你想做什么。

+

Wire

将意图编织成一条结构化的流水线——自动拆解、自动验证、自动交付。用户只审核，不操作。

=

Vibewire

一句话需求，端到端交付。人负责决策，AI 负责执行和验证。

核心理念

用户只做两件事：提需求、审结果

设计原则

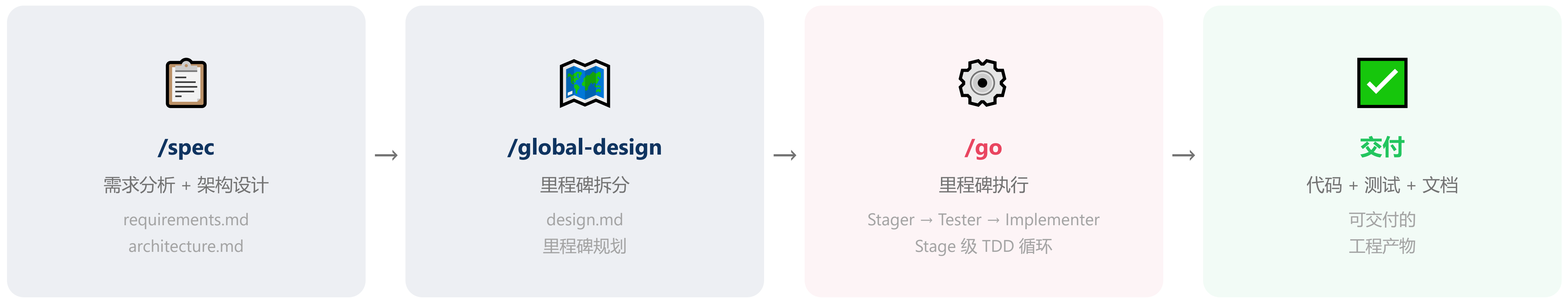
可靠性优先于速度，可追溯优先于简洁

目标

让 AI 按工程规范系统化交付

三阶段，端到端

从一句话需求到可交付代码，三个阶段依次推进，每个阶段都有明确的输入、输出和验证门。



文档驱动

每个阶段都有结构化文档输出，状态持久化在文件系统中，不依赖上下文窗口

验证门机制

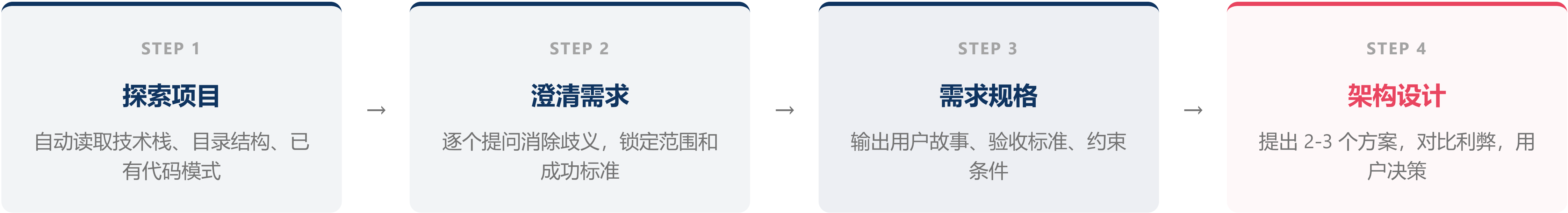
阶段之间设有人工审批点，确保方向正确后才进入下一步

Stage 级 TDD

每个 Stage 是一个完整的 RED → GREEN → REVIEW 循环，API 契约先行

/spec: 从模糊想法到精确蓝图

用户只需描述想要什么，AI 负责澄清需求、探索方案、设计架构。



设计原则

- 问题逐个问，不一股脑列清单
- 先理解项目再提方案，避免空中楼阁
- 每个方案附明确利弊，由用户决策
- 文档持久化，不受上下文窗口限制

输出产物

-  **requirements.md**
需求规格文档
-  **architecture.md**
架构设计文档
-  **spec-session.md**
规格对话总结

/global-design：从蓝图到可交付路径

将架构拆解为多个独立可交付的里程碑，每个里程碑内再拆解为渐进式阶段。



/go: 全自动执行引擎

三个专职 Agent 自动调度，每一步同时产出代码和过程记录。



核心机制

像 Git 一样**记录变化**，而非追求文档与代码的实时同步——每个 Stage 的设计决策、测试结果、实现偏差、审查修复都被完整记录

Stager 读取前序执行日志
→ 计划基于真实反馈持续演进

GREEN 不是终点

测试通过后，三维度自动审查 + 修复闭环，确保代码质量经得起真实工程考验。



同样的痛点，不同的解法

现有工具提供了好的零部件，Vibewire 提供一台完整运转的机器。

痛点	现有工具的做法	Vibewire 的解法
Spec Drift 文档与代码脱节	产出静态 Spec 文档，执行后不再同步； 靠人工维护一致性	像 Git 一样记录变化而非维持同步——每个 Stage 的设计决策、测试结果、实现偏差、审查修复都有完整日志，Stager 读取前序日志持续演进
碎片化 环节需手动串联	规划、执行、质量各是独立 Skill / Wave，用户负责衔接	一条管线自动流转： /spec → /global-design → /go ，专职 Agent 自动调度，无需人工衔接
人是操作者 用户仍需管理流程	减少工作量但不减参与度——仍需触发每步、检查状态	用户只在关键节点审批（里程碑拆分、架构方案），Agent 自动完成需求澄清、设计、测试、实现、审查、修复
质量附加 审查是后置环节	TDD / 代码审查作为可选步骤，发现问题后缺乏修复闭环	质量内建于流水线：Stage TDD + 三维度自动审查 + 自动修复闭环，审查发现问题后自动修复并验证
大项目失控 上下文耗尽	依赖对话上下文，文件外部化缓解但不解决根本问题	文件驱动 + 里程碑独立上下文：每个 Stage 由独立 Agent 在独立上下文执行，状态持久化在文件系统，跨里程碑可追溯
理解债务 发布不理解的代码	依赖用户自行审查理解，工具不强制代码可解释性	API 契约先行 + 结构化文档 ：每个 Stage 都有精确的函数签名、类型定义和测试规格，文档即代码的真实蓝图

用户只做决策，其余全自动

从需求理解到代码交付，一条管线自动流转。用户的三个审批点是唯一的暂停。



THANK YOU

Vibewire

Vibe + Wire

让 AI 自主完成从需求到交付的完整工程

项目地址

github.com/vibewire

技术栈

Claude Code Plugin · Multi-Agent

日期

2026.04